The integrity of the data in the database is a key concern. One potential cause of problems occurs when a transaction is started but a system problem prevents its completion. The result would be a database in an undefined state. The DBMS should have a built-in feature that prevents this from happening. As with all systems, regular backup is a requirement. The DBA will be responsible for backup of the stored data.

Discussion Point:

How many of the above concepts are recognisable in your experience of using a database?

11.07 Structured Query Language (SQL)

SQL is the programming language provided by a DBMS to support all of the operations associated with a relational database. Even when a database package offers high-level software tools for user interaction, they create an implementation using SQL.

Data Definition Language (DDL)

Data Definition Language (DDL) is the part of SQL provided for creating or altering tables. These commands only create the structure. They do not put any data into the database.

The following are some examples of DDL that could be used in creating the database designed in Worked example 11.02 for the theatrical agency:

```
CREATE DATABASE BandBooking;
CREATE TABLE Band (
    BandName varchar(25),
    NumberOfMembers integer);
ALTER TABLE Band ADD PRIMARY KEY (BandName);
ALTER TABLE Band-Booking ADD FOREIGN KEY (BandName REFERENCES
Band(BandName);
```


These examples illustrate a number of general points regarding the writing of SQL.

- The SQL consists of a sequence of commands.
- Each command is terminated by ;
- A command can occupy more than one line.
- There is no case sensitivity.
- There has been a decision made here to use upper case for the commands and lower case for table names, attribute names and datatypes.
- When a command contains a list of items these are separated by a comma.
- For the CREATE TABLE command this list is enclosed in parentheses

These examples show that once the database has been created, the tables can be created and the attributes defined. It is possible to define a primary key and a foreign key within the `CREATE TABLE` command but the `ALTER TABLE` command can be used as shown (it can also be used to add extra attributes).

When an attribute is defined, its data type must be specified. As with procedural languages there can be different data types or different names for data types depending on which DBMS is being used. One feature common to all databases is that the number of characters allowed for an attribute can be defined by including the number in brackets. In the above example `BandName varchar(25)` allows up to 25 characters for the band name.

The following list shows some of the names that might be used to define a data type: character, varchar, boolean, integer, real, date, time. In this chapter these will be written in lower case, but you might see them written in upper case in other sources.

TASK 11.03

For the database defined in Worked Example 11.02, complete the DDL for creating the four tables. Use varchar(8) for BookingID, integer for NumberOfMembers, date for Date, character for Headlining and varchar(25) for all other data.

Data Manipulation Language (DML)

There are three categories of use for Data Manipulation Language (DML)

- The insertion of data into the tables when the database is created
- The modification or removal of data in the database
- The reading of data stored in the database

The following illustrate the two possible ways that SQL can be written to populate a table with data:

```
INSERT INTO Band ('ComputerKidz', 5);
INSERT INTO Band-Booking (BandName, BookingID)
VALUES ('ComputerKidz', '2016/023');
```

The first example shows a simpler version that can be used if the order of the attributes is known. The second shows the safer method; the attributes are defined then the values are listed. The following are some points to note.

- Parentheses are used in both versions.
- A separate INSERT command has to be used for each tuple in the table.
- There is an order defined for the attributes.
- Although the SQL will have a list of INSERT commands the subsequent use of the table has no concept of the tuples being ordered.

The main use of DML is to obtain data from a database using a query. A query always starts with the `SELECT` command. Two examples are:

The simplest form for a query has the attributes for which values are to be listed as output identified after SELECT and the table name identified after FROM. For example:

```
SELECT BandName FROM Band;
```

Note that the components of the query are separated by spaces.

The Band table only has two attributes. To list the values for both there are two options:

```
SELECT BandName, NumberOfMembers
FROM Band;
```

or

```
SELECT * FROM Band;
```

which uses * to indicate all attributes. Note that in the first example the attributes are separated by commas but no parentheses are needed.

It is possible to include instructions in the SQL to control the presentation of the output. The following uses ORDER BY to ensure that the output is sorted to show the data with the band names in alphabetical order.

```
SELECT BandName, NumberOfMembers
FROM Band
ORDER BY BandName;
```

In this query there is no question of duplicate entries because BandName is the primary key of the BandName table. However, in the Band-Booking table an individual value for BandName will occur many times. If a query were being used to find which bands already had a booking there would be repeated names in the output. This can be prevented by the use of GROUP BY as shown here:

```
SELECT BandName
FROM Band-Booking
GROUP BY BandName;
```

An extension of the control of the output from a query is to include a condition to limit the selected data. This is provided by a WHERE clause. The following are examples:

```
SELECT BandName
FROM Band-Booking
WHERE Headlining = 'Y'
GROUP BY BandName;
```

which produces a single output for each band that has headlined. Note how a query can have several component parts which are best presented on separate lines.

```
SELECT BandName, NumberOfMembers
FROM Band
WHERE NumberOfMembers > 2
ORDER BY BandName;
```


which excludes any duo bands.

It is possible to qualify the SELECT statement by using a function. SUM, COUNT and AVG are examples of functions that work on data held in several tuples for a particular attribute and return one value. For this reason, these functions are called aggregate functions. As an example, the following code displays the number of members in a band:

```
SELECT Count(*)
FROM Band;
```

This is a special case because there is no need to specify the attribute. An example using a specific attribute would be:

```
SELECT AVG(NumberOfMembers)
FROM Band;
```

another example is:

```
SELECT SUM(NumberOfMembers)
FROM Band;
```

A query can be based on a 'join condition' between data in two tables. The most frequently used is an inner join which is illustrated by:

```
SELECT VenueName, Date
FROM Booking
WHERE Band-Booking.BookingID = Booking.BookingID
AND Band-Booking.BandName = 'ComputerKidz';
```

The SQL uses the full definitive name for each attribute with the table name and attribute name separated by a dot. The query contains two conditions. The way that the query works is as follows.

- The Band-Booking table is searched for instances where the BandName is ComputerKidz.
- For each instance the BookingID is noted.